Object Oriented Programming (CSEG1044)
Unit 02 – Lecture 02 Notes
Constructors, ‘this‘, Overloading, ‘static‘, and GC Overview

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Constructors and constructor chaining . . . 3

7.2 `this` vs `this(...)` . . . 4

7.3 Method overloading (rules) . . . 4

7.4 `static` keyword . . . 4

7.5 Garbage collection overview . . . 5

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: `Employee` **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **7**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces
- **Main new ideas:** Constructors and constructor chaining, `this` vs `this(...)`, Method overloading (rules)
- **Course thread:** Use Constructors, 'this', Overloading, 'static', and GC Overview to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

2 Lecture Overview

This lecture explains how objects are created and how to design classes cleanly:

- constructors and constructor chaining,
- `this` vs `this(...)`,
- method overloading,
- the `static` keyword,
- and a realistic overview of garbage collection (GC).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Class state and behavior**: Constructors and overloading only make sense once fields and methods already mean something in one class. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Reference semantics**: Constructors make object creation explicit, not just primitive assignment. Main reference: `Unit 02 / Lecture 01 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Write constructors and use `this(...)` for constructor chaining.
- Overload methods correctly and avoid common mistakes.
- Use `static` for class-level state/behavior and explain GC basics.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Constructors and constructor chaining* → ***`this`*** *vs* ***`this(...)`*** → *Method overloading (rules)* → ***`static`*** *keyword* → *Garbage collection overview.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Constructor**: special method to initialize an object (same name as class, no return type).
- **Shadowing**: parameter/local variable has same name as field (e.g., `name`).
- `this`: reference to the current object.
- `this(...)` : call another constructor in the same class (must be first statement).
- **Overloading**: same method name, different parameter list (compile-time selection).
- `static`: belongs to the class, shared across all objects.
- **Garbage collection**: automatic memory reclamation for unreachable objects.

7 Detailed Notes

7.1 Constructors and constructor chaining

Constructors are used to:

- put an object into a valid initial state,
- enforce invariants early (validate inputs),

- and set default values when necessary.

Default constructor rule: If you do not write any constructor, Java provides a default no-arg constructor. If you write *any* constructor, Java does *not* automatically create a default one.

Constructor chaining:

- Use `this(...)` to call another constructor in the same class.
- `this(...)` must be the **first statement** in the constructor.

7.2 `this` vs `this(...)`

- `this` refers to the current object (useful for field access and method calls).
- `this(...)` calls another constructor.

Shadowing example: if constructor parameter is `name`, the field is also `name`. Use `this.name = name;` to distinguish.

7.3 Method overloading (rules)

Overloading means:

- same method name,
- different parameter list (number/type/order).

Not overloading:

- changing only the return type,
- changing only parameter names.

Why it matters. Overloading lets you offer "one concept" in multiple forms (e.g., `print(int)` and `print(String)`).

7.4 `static` keyword

Static field is shared among all objects:

- Use it for counters, constants, or shared configuration.

Static method belongs to the class:

- You can call it as `ClassName.method()`.
- It cannot directly access instance fields (because there is no `this`).

Utility class pattern: when a class only has static methods (like `Math`), make the constructor `private` to prevent creating objects.

7.5 Garbage collection overview

GC frees memory of objects that are **no longer reachable** from any live reference. Key points:

- You cannot reliably force GC (`System.gc()` is only a suggestion).
- GC does not prevent all "memory leaks" if references are kept unnecessarily (e.g., in a static list).
- GC improves safety compared to manual memory management, but you still must close files, DB connections, etc.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

8 Running Project Connection

Use Constructors, 'this', Overloading, 'static', and GC Overview to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Using this and this() as if they solve the same problem.
- Trying to overload by return type alone.
- Putting per-object data inside a static field.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

10 Worked Example: `Employee`

```
public class Employee {
    private static int nextId = 1;
```

```java
    private final int id;
    private final String name;
    private double salary;

    public Employee() {
        this("NA", 0.0); // constructor chaining
    }

    public Employee(String name, double salary) {
        if (name == null || name.isBlank()) throw new IllegalArgumentException("name");
        if (salary < 0) throw new IllegalArgumentException("salary");

        this.id = nextId++;
        this.name = name;
        this.salary = salary;
    }

    // Overloading: same name, different parameters
    public void raiseSalary(double percent) {
        raiseSalary(percent, 0.0);
    }

    public void raiseSalary(double percent, double bonus) {
        if (percent < 0) throw new IllegalArgumentException("percent");
        salary = salary + (salary * percent / 100.0) + bonus;
    }

    public static int getNextId() { return nextId; }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Constructors and constructor chaining* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. What is the difference between `this` and `this(...)`?

Answer: `this` refers to current object; `this(...)` calls another constructor.

2. Can we overload a method by changing only return type?

   Answer: No.

3. Can a static method access an instance field directly?

   Answer: No, because there is no `this`. It needs an object reference.

13 Self-Study Practice (no solutions)

1. Create a `Point` class with 3 constructors (0-arg, 2-arg, copy constructor) using chaining.
2. Overload a method `area(...)` for a square (1 param) and rectangle (2 params).
3. Write a class-level counter using a static field and print how many objects were created.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why do we use constructor chaining?

Sample answer: It reduces duplicate initialization code. One constructor can call the main constructor that performs validation and sets fields consistently.

Q2. What does garbage collection do?

Sample answer: GC reclaims memory of objects that are no longer reachable. It does not guarantee immediate collection and does not automatically close external resources like files or DB connections.

Q3. Why is constructor overloading useful?

Sample answer: It supports multiple valid initialization paths while keeping one coherent class design.